

- [Complete SIM Module Integration Guide \(Raspberry Pi + Home Assistant\)](#)
  - [Document 1: SIM Module Setup with Raspberry Pi — Production Cheat Sheet](#)
  - [Document 2: SIM Module Emergency Call & SMS Setup — Standalone Battery-Isolated Blueprint](#)

# Complete SIM Module Integration Guide (Raspberry Pi + Home Assistant)

*Merged reference combining: (1) SIM Module Setup with Raspberry Pi — MQTT/ESP32 Production Cheat Sheet, and (2) SIM Module Emergency Call & SMS Setup — Standalone Battery-Isolated Blueprint. All topics from both source documents are preserved below, in their original order, grouped by source document.*

## Document 1: SIM Module Setup with Raspberry Pi — Production Cheat Sheet

Complete A-to-Z production cheat sheet for integrating a SIM module (such as a SIM800L or SIM7600 GSM modem via UART/USB) onto a Raspberry Pi server, complete with Python code, systemd services, terminal maintenance commands, Home Assistant entities, and MQTT triggers for ESP32 nodes.

### Part 1: Hardware Connection Diagram (UART / USB)

If using a standard serial GSM module, wire it to the Raspberry Pi GPIO header or plug it into a USB-to-TTL adapter:

| Raspberry Pi GPIO     |        | SIM Module (SIM800L/7600) |
|-----------------------|--------|---------------------------|
| Pin 2 (5V / 3.3V)     | -----> | VCC (Requires stable 2A!) |
| Pin 6 (GND)           | -----> | GND                       |
| Pin 8 (GPIO 14 / TX)  | -----> | RXD                       |
| Pin 10 (GPIO 15 / RX) | <----- | TXD                       |

**Critical Note:** Most SIM modules (like SIM800L) draw sudden 2A power spikes during network registration. **Do not** power the module directly from the Raspberry Pi 5V pin, or the Pi will reboot due to voltage drops. Use an external 5V 2A power supply with **common ground** tied to the Pi.

### Part 2: Python Integration Code (/home/pi/pi\_sim/sim\_controller.py)

Create a directory /home/pi/pi\_sim/ and save the following production script. It listens to the MQTT broker, parses the SOS payload, and triggers an AT-command phone call via serial.

```
import time
import serial
from paho.mqtt import client as mqtt_client

# MQTT Broker Configuration
BROKER = "localhost"
PORT = 1883
TOPIC = "home/redwannabil/sim/cmd"
CLIENT_ID = "pi_sim_controller"

# Serial Port Configuration (Change to /dev/ttyUSB0 if using a USB modem)
SERIAL_PORT = "/serial0"
BAUD_RATE = 9600
TARGET_PHONE_NUMBER = "+8801700000000" # Replace with your emergency contact number

try:
```

```

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=5)
print("[SIM] Serial modem initialized successfully.")
except Exception as e:
    print(f"[SIM ERROR] Failed to open serial port: {e}")
    ser = None

def make_emergency_call():
    if not ser:
        print("[SIM ERROR] Modem not connected.")
        return
    try:
        print("[SIM] Forcing network detach & re-sync...")
        ser.write(b'AT+CFUN=1,1\r')
        time.sleep(3)
        print(f"[SIM] Dialing emergency number {TARGET_PHONE_NUMBER}...")
        ser.write(f'ATD{TARGET_PHONE_NUMBER};\r'.encode())
        time.sleep(20) # Keep line open for call duration
        print("[SIM] Terminating call...")
        ser.write(b'ATH\r')
    except Exception as e:
        print(f"[SIM ERROR] Call failed: {e}")

def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        print("[MQTT] Connected to Broker, subscribing to topic...")
        client.subscribe(TOPIC)
    else:
        print(f"[MQTT] Failed to connect, return code {rc}")

def on_message(client, userdata, msg):
    payload = msg.payload.decode()
    print(f"[MQTT] Received command: {payload} on topic {msg.topic}")
    if payload == "SOS":
        make_emergency_call()

def run():
    client = mqtt_client.Client(mqtt_client.CallbackAPIVersion.VERSION1, CLIENT_ID)
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(BROKER, PORT)
    client.loop_forever()

if __name__ == '__main__':
    run()

```

## Part 3: Systemd Service Configuration

To run the Python SIM controller automatically in the background on system boot, create a systemd service file.

1. Create the service file using nano:

```
sudo nano /etc/systemd/system/pi_sim.service
```

2. Paste the exact configuration below:

```

[Unit]
Description=Raspberry Pi SIM Module MQTT Controller
After=network.target mosquitto.service

[Service]
ExecStart=/usr/bin/python3 /home/pi/pi_sim/sim_controller.py
WorkingDirectory=/home/pi/pi_sim/
StandardOutput=inherit
StandardError=inherit
Restart=always
RestartSec=10
User=pi

[Install]
WantedBy=multi-user.target

```

3. Save and exit (Ctrl+O, Enter, Ctrl+X), then enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable pi_sim.service
sudo systemctl start pi_sim.service
```

## Part 4: Terminal Maintenance & Debugging Commands

Use these exact commands to monitor, restart, or troubleshoot the service on the Raspberry Pi terminal:

- **View live logs in real-time (Follow mode):**

```
sudo journalctl -u pi_sim.service -f
```

- **Check service health and recent exit status:**

```
sudo journalctl -u pi_sim.service -n 50 --no-pager
```

- **Restart the SIM service:**

```
sudo systemctl restart pi_sim.service
```

- **Stop the service:**

```
sudo systemctl stop pi_sim.service
```

- **Check current service status (Active / Dead):**

```
sudo systemctl status pi_sim.service
```

- **Test MQTT command manually via terminal (Simulates ESP32 trigger):**

```
mosquitto_pub -h localhost -t "home/redwannabil/sim/cmd" -m "SOS"
```

## Part 5: Home Assistant Integration & Dashboard Entities

To monitor and manually trigger the SIM module straight from the Home Assistant dashboard, add this manual MQTT Switch configuration to configuration.yaml (or an mqtt.yaml split file):

```
mqtt:
  switch:
    - name: "Pi Server SIM Emergency Dialer"
      command_topic: "home/redwannabil/sim/cmd"
      payload_on: "SOS"
      payload_off: "OFF"
      state_topic: "home/redwannabil/sim/state"
      optimistic: true
      icon: mdi:phone-alert
```

**How to use it on the Dashboard:** Add a standard **Entity Card** or **Button Card** and link it to switch.pi\_server\_sim\_emergency\_dialer. Tapping it will instantly publish the sos payload to MQTT, executing a phone call via the Pi.

## Part 6: Future ESP32 MQTT Broker Integration & Automations

When ESP32 nodes (or an ESP32 Master Center Node) connect to the Home Assistant Mosquitto MQTT broker, they can publish sensor metrics or listen for emergency commands seamlessly.

### 1. ESP32 Arduino Code Snippet (Publishing an Emergency State to MQTT)

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* mqtt_server = "192.168.1.100"; // IP address of your Raspberry Pi Server
WiFiClient espClient;
```

```

PubSubClient client(espClient);

void sendEmergencySOS() {
  if (client.connected()) {
    client.publish("home/redwannabil/sim/cmd", "SOS");
    Serial.println("[MQTT] Emergency SOS signal dispatched to Pi Server!");
  }
}

```

## 2. Full Home Assistant Automation Template (Linking ESP32 Sensors to the SIM Dialer)

This automation watches ESP32 sensors (like temperature or earthquake sensors) and automatically fires the MQTT command to the Pi SIM module when danger is detected:

```

alias: "📞 Master Emergency: ESP32 Trigger to Pi SIM Dialer"
description: "Triggers phone call via Pi SIM controller when ESP32 detects a critical ha
mode: single
triggers:
  - trigger: numeric_state
    entity_id: sensor.server_room_temperature
    above: 50
actions:
  - action: mqtt.publish
    data:
      topic: "home/redwannabil/sim/cmd"
      payload: "SOS"
      qos: 1

```

## Document 2: SIM Module Emergency Call & SMS Setup — Standalone Battery-Isolated Blueprint

Complete, standalone blueprint for a SIM Module Emergency Call & SMS Setup. This guide covers the physical wiring (including the safe TP4056 lithium battery isolation system), the Python code updates, and the Home Assistant configuration.

### Hardware Architecture & Safety Summary

- **GSM Module:** Standard SIM800L (or compatible cellular breakout).
- **Power Engine:** 1× brand-new, high-quality 18650 Li-ion battery (degraded/old batteries will sag and fail during transmission spikes).
- **Charging IC:** TP4056 Lithium Charging Module.
- **Safety Isolation Strategy:** The Raspberry Pi 5 runs a steady 5V current into the TP4056 input. The SIM module is wired directly to the battery/charger outputs in parallel. This fully shields the Pi from the module's massive 2 Amp transmission bursts while maintaining a safe, unified ground loop.

⚠️ **CRITICAL ELECTRICAL WARNING:** Never bridge the 3.7V battery's positive terminal directly to any 5V or 3.3V GPIO pin on the Raspberry Pi. Doing so will overcharge the battery (causing fire/explosion risks) or instantly kill the Pi's internal voltage regulators.

### Phase 1: Physical Wiring Diagram & Pinouts

Ensure all connections are soldered or pinned securely according to this roadmap:

**1. The Power & Charging Loop** - Pi 5V (Physical Pin 2 or 4) → Connect to IN+ on the TP4056 board. - Pi GND (Physical Pin 6) → Connect to IN- on the TP4056 board. - Battery Positive (+) → Connect to B+ (or BAT+) on the TP4056. - Battery Negative (-) → Connect to B- (or BAT-) on the TP4056. - SIM Module VCC → Connect directly to the B+ pad on the TP4056 (parallel with the battery). - SIM Module GND → Connect directly to the B- pad on the TP4056 (parallel with the battery).

**2. The Serial Data Loop** - SIM Module TXD → Connect to Pi RXD (GPIO 15 / Physical Pin

10). - SIM Module RXD → Connect to Pi TXD (GPIO 14 / Physical Pin 8). - **Note:** Because the Pi's ground meets the charger at IN-, and IN- shares an internal trace with B-, the Pi and SIM module automatically establish the required common ground loop.

## Phase 2: Raspberry Pi OS Serial Port Preparation

By default, Debian uses the hardware serial port for an operating system login terminal. This must be disabled so the custom Python script can control the port.

1. SSH into the Raspberry Pi and run:

```
sudo raspi-config
```

2. Navigate to **Interface Options** → **Serial Port**.
3. When asked: "Would you like a login shell to be accessible over serial?" → Select **No**.
4. When asked: "Would you like the serial port hardware to be enabled?" → Select **Yes**.
5. Save, exit, and reboot the system:

```
sudo reboot
```

## Phase 3: Complete Python Automation Script

This script includes the original telemetry loop, the automatic sensor watchdog triggers, and a new local HTTP Webhook server that processes manual SOS commands sent from Home Assistant.

Create or update the script file (e.g., nano ~/security\_system.py):

```
# -*- coding: utf-8 -*-
import time
import math
import datetime
import subprocess
import threading
import serial
import requests
import smtplib
from http.server import BaseHTTPRequestHandler, HTTPServer
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# =====
# SYSTEM & EMERGENCY CONFIGURATION
# =====
HA_URL = "http://192.168.0.40:8123"
HA_TOKEN = "YOUR_HA_LONG_LIVED_ACCESS_TOKEN"
TELEGRAM_BOT_TOKEN = "YOUR_TELEGRAM_BOT_TOKEN"
TELEGRAM_CHAT_ID = "YOUR_TELEGRAM_CHAT_ID"
EMERGENCY_NUMBERS = ["+8801794684164", "+8801342570575"]

GMAIL_USER = "nabilredwoan2005@gmail.com"
GMAIL_APP_PASSWORD = "uexpklsjjfuqxhkt" # Clean app password space
ALERT_RECIPIENTS = ["redwannabil16@gmail.com"]

# =====
# HARDWARE PORT INITIALIZATION
# =====
# Uses the hardware mapped port /dev/serial0
sim = serial.Serial('/dev/serial0', 9600, timeout=1)
alarm_active = False

# =====
# SIM MODULE CRITICAL EMERGENCY CORE
# =====
def emergency_sos(message):
    """Executes consecutive SMS alerts and live call sequences over AT commands"""
```

```

for number in EMERGENCY_NUMBERS:
    try:
        print(f"[SOS] Targeting: {number}")
        # 1. Initialize text message frame
        sim.write(b'AT+CMGF=1\r\n')
        time.sleep(0.5)
        # 2. Stage destination directory routing
        sim.write(f'AT+CMGS="{number}"\r\n'.encode())
        time.sleep(0.5)
        # 3. Write payload text and append Hex 1A (Ctrl+Z) to send
        sim.write(message.encode() + b'\x1a')
        time.sleep(4)
        print(f"[SOS] SMS dispatched to {number}. Initializing voice call link...")
        # 4. Dial emergency route line
        sim.write(f"ATD{number};\r\n".encode())
        time.sleep(20) # Keep line open for 20 seconds to force loud ring vibration
        # 5. Terminate voice channel link
        sim.write(b"ATH\r\n")
        time.sleep(1)
    except Exception as e:
        print(f"[SIM ERROR] Failed executing emergency pipeline for {number}: {e}")

def reset_alarm_lock():
    global alarm_active
    time.sleep(60) # Protects system from duplicate trigger looping within 1 minute
    alarm_active = False

def trigger_full_alarm(message):
    global alarm_active
    if not alarm_active:
        alarm_active = True
        print(f"[ALARM ACTIVATE] Reason: {message}")
        threading.Thread(target=reset_alarm_lock, daemon=True).start()
        threading.Thread(target=emergency_sos, args=(message,), daemon=True).start()

# Add background threads here for Telegram, Gmail, and Buzzers as needed

# =====
# HOME ASSISTANT MANUAL SOS WEBHOOK INTERFACE
# =====
class SOSWebHookHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        if self.path == '/trigger_manual_sos':
            msg = "⚠️ MANUAL EMERGENCY SOS: Your home person is in danger! Please take
immediate action!"
            trigger_full_alarm(msg)
            self.send_response(200)
            self.send_header('Content-type', 'application/json')
            self.end_headers()
            self.wfile.write(b'{"status": "activated"}')
        else:
            self.send_response(404)
            self.end_headers()

    def log_message(self, format, *args):
        return # Suppresses console output floods

def run_webhook_server():
    server = HTTPServer(('0.0.0.0', 8089), SOSWebHookHandler)
    print("[SERVER] SOS Webhook Core active on port 8089...")
    server.serve_forever()

# Launch interface thread
threading.Thread(target=run_webhook_server, daemon=True).start()

# =====
# MAIN EXECUTION HOLD
# =====
if __name__ == "__main__":
    while True:
        time.sleep(1)

```

## Phase 4: Home Assistant Integration

Link the dashboard interface to the script's incoming server port.

### Step 1: Establish the Shell Target Command

1. Open the Home Assistant system folder setup:

```
nano ~/homeassistant/config/configuration.yaml
```

2. Find or create the top-level `shell_command:` tree entry and register the network trigger:

```
shell_command:  
  trigger_manual_sos: "curl -X POST http://127.0.0.1:8089/trigger_manual_sos"
```

3. Save, close, and restart the Home Assistant container instance.

### Step 2: Render the Safe Dash Card

Open the Home Assistant UI dashboard environment, choose **Add Manual Card**, and use this specific structure:

```
type: button  
name: 🚨 TRIGGER EMERGENCY SOS 🚨  
icon: mdi:alert-octagon  
icon_height: 80  
show_name: true  
show_icon: true  
show_state: false  
tap_action:  
  action: none  
hold_action:  
  action: call-service  
  service: shell_command.trigger_manual_sos
```

**Note:** Binding this to `hold_action` ensures a full 2-second press-and-hold on the button card is required to fire the alarm, preventing accidental taps while scrolling the phone UI.